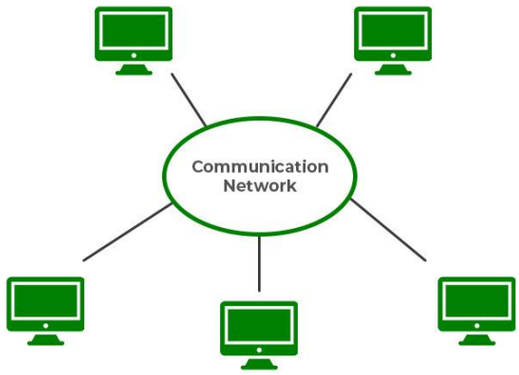
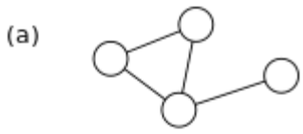
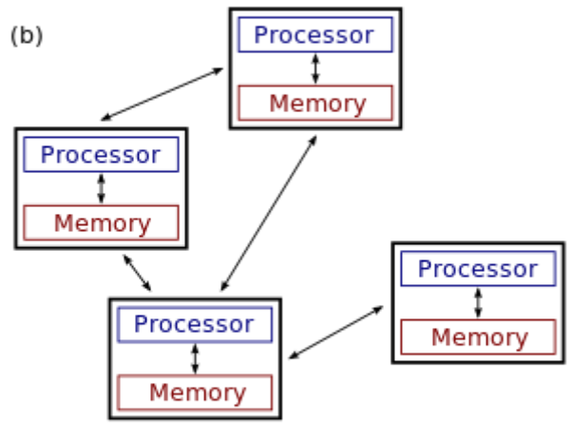
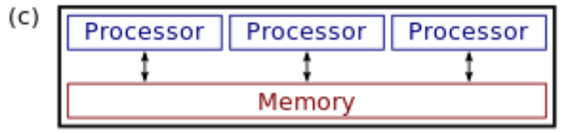








Distributed Systems

- A collection of interconnected computers that work together to achieve a common goal.
- Can also be a collection of running programs that work together. E.g. game client + cheat detector, windows application + notification handler.
- Creating distributed systems would require a combination of parallel computing and networking techniques.
- The entities communicate with each other by message passing → glorified chat.
- Internet is a huge distributed system.

Applications of distributed systems

Finance and commerce	eCommerce, e.g., Amazon and eBay, PayPal, online banking and trading
The information society	Web information and search engines, ebooks, Wikipedia; social networking: Twitter, Instagram, Snapchat.
Creative industries and entertainment	online gaming, music and film in the home, user-generated content, e.g., YouTube, Twitch, Steam, Playstation Network
Healthcare	health informatics, on online patient records, monitoring patients
Education	e-learning, virtual learning environments, e.g., Canvas; distance learning
Transport and logistics	GPS in route finding systems, map services: Google Maps, Google Earth, Waze
Science	The Grid as an enabling technology for collaboration between scientists
Environmental management	sensor technology to monitor earthquakes, floods or tsunamis

Low-key examples of distributed systems:

- Mobile development IDE - Mobile devices with USB debugging

- Google Home, IOT
- Swarm coordination. E.g. drones.
- Web updates from a game. E.g. Progress trackers, character inventory viewers.

Observation: Any program, that requires communication with other programs, require distributed computing techniques.

Challenges when designing distributed systems

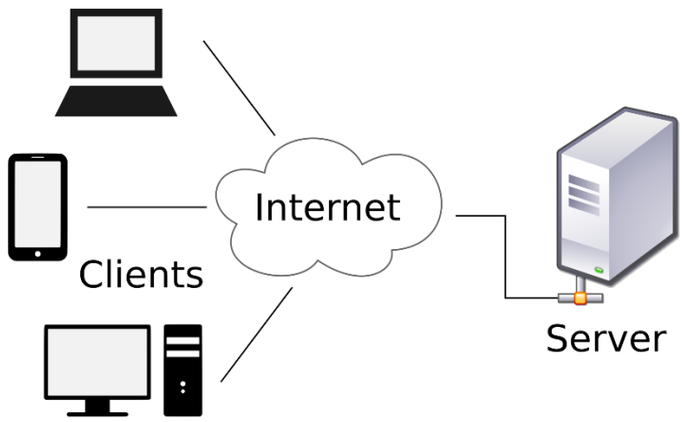
1. Concurrency – since multiple programs, across different hardware, different locations, are required, then concurrency techniques are needed.
2. Tolerance to failures – when one program fails, there is a risk of cascading failures – the cluster fails. One must design a fault-tolerant distributed system.
3. Scalability – Can we easily add new clusters to further improve the performance of the distributed system?
4. Concealment – A user must perceive the distributed system as only 1 whole program, rather than a collection of independent components.
5. Quality of service – how fast can the distributed system provide results?

Got distributed system skills? What are the possible jobs?

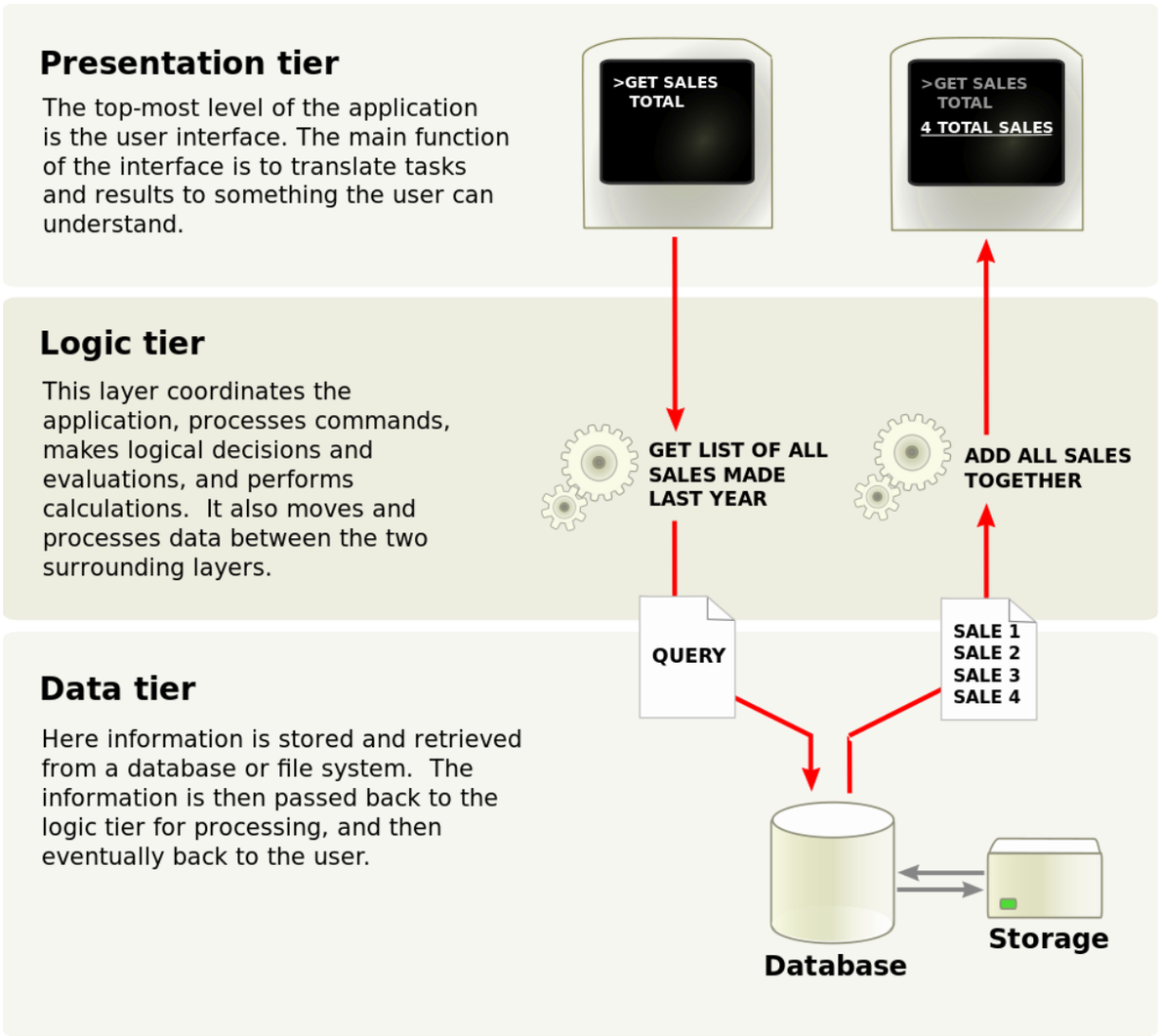
- File storage systems for servers → Google Drive
- Cloud compute → Amazon Web Servers, Google Cloud
- Real-time services → Banking transactions, MMO services

System models

There can be many ways to design distributed systems. It all depends on the problem and the best solution required. Common system models are: client-server, three-tier, n-tier, peer-to-peer.

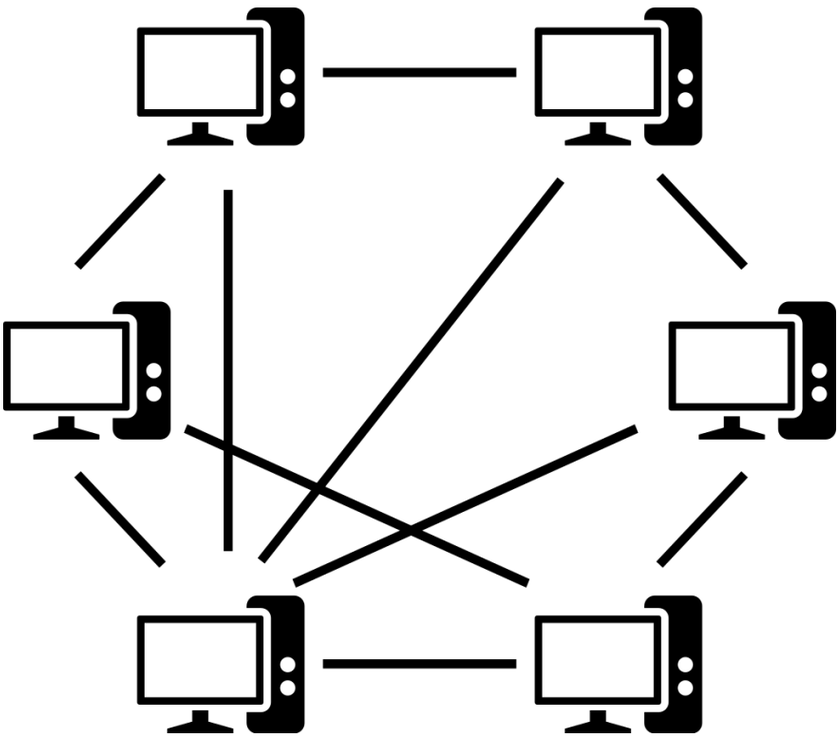


Client-server architecture is the most common. Server handles most of the demanding computations, clients generally handle light-weight computations, or generally receive the results from the server. Major cons of this is the server is a possible single point of failure.



The **three-tier** architecture attempts to separate the presentation/viewing processes, the logical operations, and the data storage in three different groups. Each tier can have its own collection of entities.

The **N-tiers** introduces additional tiers, aside from presentation, logic and data tiers.



The peer-to-peer architecture doesn't have a server. Each client also acts as its own server. A portion of computing resources is allocated for processing and sending/receiving data from other network participants. Good example: torrents.

Inter-process Communication
Basic idea: Pass messages between applications. Message passing is the core operation for all distributed systems.

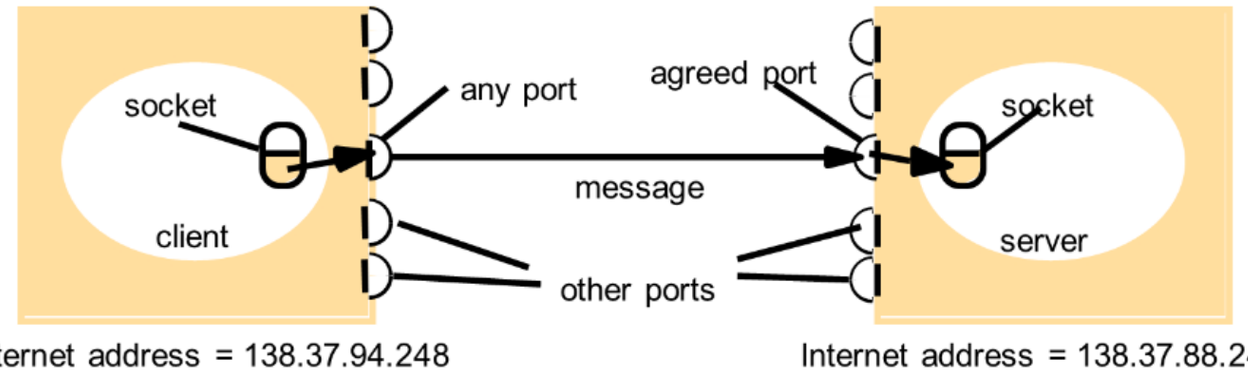
You can do a lot with message passing:

- Chat system
- Send images, videos through file streams.
- Trigger remote operations

Different libraries are available for supporting inter-process communications for distributed systems: Elixir, Netflix Hystrix, Resilience4J, LizardFS.

Sockets
For a process to receive messages:

- It's a socket must be bound to a local port and one of the internet addresses of the computer on which it runs.
- Processes may use the same socket for sending and receiving messages → there can be 655365 possible port numbers to use: https://networkappers.com/tools/open-port-checker
- Any process may make use of multiple ports to receive messages, but a process cannot share ports with other processes on the same computer.



Most Common Ports

- 21 FTP
- 22 SSH
- 23 TELNET
- 25 SMTP
- 53 DNS
- 80 HTTP
- 110 POP3
- 115 SFTP
- 135 RPC
- 139 NetBIOS
- 143 IMAP
- 194 IRC
- 443 SSL
- 445 SMB
- 1433 MSSQL
- 3306 MySQL
- 3389 Remote Desktop
- 5632 PCAnywhere
- 5900 VNC
- 3724 WOW
- 9200 Elastic Search
- 6379 Redis
- 5432 Postgre SQL
- 1521 Oracle
- 1433 MSSQL
- 3306 MariaDB
- 5000 .net Local
- 987 PlayStation Wake On Lan
- 989 FTPS
- 995 Post Office Protocol

Hands-on activities

- Demonstration of simple chat system using C++ sockets. GDPARCM_HO7-SocketsDemo.